Execution Time Prediction For Serverless Workload

Yunxin Sun
Dept. of Computer Science
ETH Zürich
yunsun@ethz.ch

Yiqun Liu
Dept. of Computer Science
ETH Zürich
yiqliu@ethz.ch

I. Motivation

Serverless computing, also known as "Function as a Service"(FaaS) is a new paradigm for cloud computing. In this kind of cloud computing, the user does not need to consider the underlying infrastructure and dramatically simplifies the development procedure. The cloud provider could also have more flexibilities to schedule serverless functions on different servers for better resource utilization than IaaS or PaaS. The serverless function is usually short-lived, and the scheduling policy should automatically scale up or down to meet the application demands. Due to the above reasons, both the academy and the industry has had a great interest in designing scheduling policies for the serverless functions because the serverless function has very different characteristics than other cloud workloads. It is beneficial for the scheduler to know the resource requirements(by predicting) and additional related information in advance.

In this project, we focus on predicting the total execution time of a particular serverless function. Suppose the scheduler knows the execution time in advance. In that case, it can locate the execution environment of that serverless function on specific servers based on CPU usage and hopefully lead to better resource utilization. This idea is very promising. However, it also has several challenges. *First*, the execution time of a serverless function is typically short-lived. This feature poses a stricter requirement on the latency of the prediction. *Second*, serverless functions often call other cloud services, such as the cloud storage service. This practice adds a great deal of variability to the predicted execution time. *Third*, we have little knowledge of how each cloud provider implements the serverless function under the hood. As a result, it is difficult to tell whether the insight we gain is platform-dependent or fundamental.

In this distributed systems lab project, we use the Google Cloud Platform to implement all our ideas and conduct all the related experiments. We first set up a monitor pipeline to collect, clean, and analyze the metrics data about the serverless function of the Google Cloud Platform. We then move on to the central part about predicting the execution time of the serverless function. In addition to the total execution time, we also provide the prediction of the ETA(Estimated Time of Arrival). However, we observe that the execution time of the same serverless function can be distributed widely, even under the same set of function parameters, especially under low resource availabilities. We thus predict another value, the confidence of our predicted execution time, as another input to the scheduler. Finally, we talk about some limitations of this work and some possible future directions.

II. Resource Monitor and Data Collection

In industrial settings, we do not have access to serverless function code, but instead we are able to modify the runtime system, which is the right opposite of our situation. To still analyze function behavior on commercial platforms, we implemented a user-space monitoring system. Our monitoring approach is general and minimally-intrusive, and an equivalent can be easily implemented for industrial settings.

The monitoring system is composed of 1) a local python package *RM*, and 2) five lines of instrument code. Instrument code are added to function code as "header and footer", and original function body remains intact. Note that this approach is specific to research settings but the framework is general.

On deployments, *RM* are bundled and uploaded along with the monitored function. When function execution starts, monitored function will start the *RM* monitoring thread, which periodically samples resource utilization information. *RM* thread extracts more than 40 (the number depends on number of threads and sockets in use) metrics from `/proc` directory covering four aspects of resources: CPU, memory, network and disk.

When monitored function terminates, *RM* writes the samples as logs. A custom log router in GCP logging system will extract and redirect our samples to a storage bucket periodically. The complete data pipeline is shown in Fig. 1.

```
def function_handler(request):
    import resource_monitor as monitor
    monitor_thread =
        monitor.ResourceMonitorThread(
            0.1, 0.1, output=True
        )
    monitor_thread.start()

    # function code goes here
    # ......

    monitor_thread.signal_terminate()
    monitor_thread.join()
```

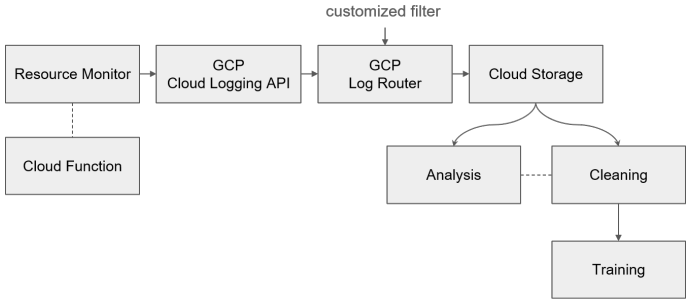


Fig. 1. Data Pipeline

III. Execution Time Prediction

To the author's knowledge, quality benchmarks on realistic serverless workloads are limited by the time of writing. Most benchmarks contain synthesis workloads and are designed to measure and compare the performance of FaaS runtime platform. Whether it reflects the intrinsic characteristics of serverless workloads is questionable. In addition, these benchmarks do not include a representative set of invocation parameters, making it difficult to infer the variation of performance in production settings.

Therefore, our design space is heavily limited by the scarcity of quality workload benchmark. We focus on predicting remaining execution time based on runtime profiling and historical execution information only. This predicting framework more general and does not heavily depend on the quality of benchmarks.

A. Workload Characterization

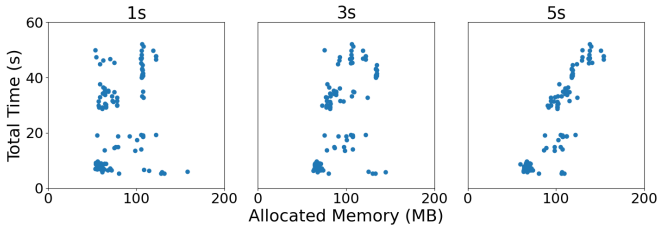


Fig. 2. Correlations Between Target and Key Features Become Obvious Over Time

We run experiments on 14 functions of FunctionBench [1] benchmark. We gather function inputs using following ways:

1) For numerical inputs, we sample from a uniform distribution.

2) For image file inputs, we randomly selects images from Google Open Images Dataset [3].

3) For video file inputs, we randomly selects videos from HMDB51 [4]. Videos are randomly concatenated to increase input variations.

In all cases, inputs which causes timeout or OOM errors are not considered. We found that the workloads can be naturally grouped into three categories, as shown in table I. For workloads which demonstrate low execution time variance within our experiments, simple memoization-based approach could provide sufficiently accurate predictions.

TABLE I
Workload Taxonomy

Execution Time	Low Variance	High Variance
Short-lived	float, chameleon matmul, payes	-
Long-lived	nlp-prediction	image processing video processing linpack

[a]Only successful invocation are considered. (ignore timeout, OOM)

For long-lived workloads which has a high execution-time variance, we analyzed the correlations between collected data (utilization information) and execution time, and has following findings:

1) peak memory usage during an invocation time is highly correlated with execution time

2) for workloads which process an input file, the amount of data downloaded at their initial phase is highly correlated with execution time

3) variations in execution is higher when the workloads involves download from network, or when the execution time is longer

One possible explanation to 1) and 2) is that both of these metrics reflects the input size or the size of working set of the function. However, none of these metrics are available before execution, and we are facing a timeliness-accuracy tradeoff: the more we wait, more likely we observe the peak value, but less timeliness the predictions can be, as shown in Fig. 2.

B. Predicting System

Since network activities or memory allocation event could expose the size of input, we build a ML system to estimate total execution time on the fly. The input to the ML system is the view of the function from the perspective of the resource manager, at certain moments. Metrics collected by resource monitor are used as input, which includes both instant metrics, such as memory usage and CPU utilization, and also some accumulated values, such as data downloaded, CPU time. The output is the total execution time of the function, from which the remaining execution time can easily calculated. The system is designed to be queried at any time during function execution. A scheduler can choose co-locate workload based on this information to avoid resource fragmentation.

Several machine learning models are evaluated. Since optimizing models is not a focus of this project, we simply use out-of-the-box implementation in *Scikit-learn* [2] library, and train them using mean square error as the cost functions.

C. Data Cleaning

We identify a few synonym which leads to outliers on key metrics.

1) GCP memory allocation optimization: When experiments are run repetitively, memory allocation even cannot be captured by our resource monitor for about 40% of the invocations. We conjecture that this might an optimization on memory management system: the new function will reuse an existing executor process with enough memory allocated. In this case, the memory is neither recycled no re-allocated. The working set information is hidden for us completely. Since memory information is a key to our prediction, we always drop this kind of samples.

2) GCP gUnicorn initialization: Repeated experiments show that about less than 20% of the invocations has significantly less execution time, and we found that the outliers all come together with a "gUnicorn initialization" GCP log. Therefore, we conjecture that it is possibly because the resource is over-provisioned in this case. We identify these outliers through repeated experiment, in later section we compare the accuracy training with and without this type of data cleaning.

D. Data Augmentation

We use our insights from workload characterization to augment input features. Historical data is enriched in two ways: 1) the minimum and maximum observed in current execution 2) the timestamp when peak value is reached or when fastest increase in that counter is reached. We expect these features to encode more information of the key events.

E. Evaluations and Observations

We evaluate our method on multiple models and compare the results with and without proposed optimization. The experiments are run on our custom dataset. 80% of the samples are used for training while 20% of the samples are left as test set. The results are summarized in table II, where the best result for each workload is highlighted.

For the three workloads shown in the table, image processing applies traditional image transformation, video processing make color conversions on each frame of the videos, and linpack solves a linear equation systems of particular size. image processing and video processing reads input file from cloud storage. To verify our assumption of timeliness-accuracy tradeoff, we use the observations at 1s, 3s and 5s after spin-up to train the models.

The first half of table II shows the error averaged across all models, it demonstrates how obvious the the correlation is in general. The second half contains the errors of the best model for that specific workload. The accuracy significantly improved with our data augmentation, but whether data cleaning is beneficial remains in doubt. There are two possible explanation: 1) The dataset is so small that denoising do not make a significant difference at this accuracy level 2) the model could effectively recognize gUnicorn events and made predictions accordingly. In addition, the quality of predictions improves over time. This verifies our conjecture. With more time to observe resource utilization, the monitor could observe more information and thus makes better predictions. However, the improvement do not continues indefinitely: for workloads in our test set, predictions made at 15s are no better than those made at 10s. We argue that the improvement is powered by key events the monitor observed, such as download and memory allocation. Time itself is not the driving factors.

And for the three workloads tested, network and memory are the most important information source (with respect to correlation with execution time). Compared to memory activity, network activity is more timely, but less accurate and less available (e.g. workloads like *linpack* does not involves network transfer).

F. Limitations

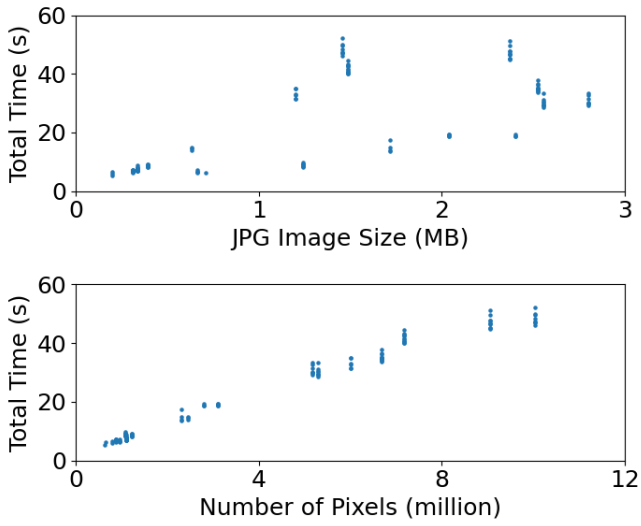


Fig. 3. JPG Compression Hides True Input Size and Obscures Correlation

We expect the application of our predicting system is limited by its limitations in accuracy and applicability:

Accuracy Limitations: Many factors limit the accuracy our predicting system: 1) the performance variations of serverless platform: for heavy workloads, we observe a coefficient of variation up to 5%. 2) the intrinsic predicting difficulties of the workload: some functions are essentially hard to predictable, maybe due to their control flow. But even for predictable workloads, file encoding might obscure the correlation between target and key features. Fig. 3 demonstrates this effect. Figures below presents the strong linear correlation between our target to image size (number of pixels). However, images are typically compressed according to standards like JPEG. In this case, file size is a inferior indicator of input size. Encoding-aware data augmentation might be helpful, but this might have regulation issues and is not always applicable. 3) the limited size of dataset and little model optimization done to our models (not a focus of this project).

Applicability Limitations: 1) this predicting framework makes prediction on-the-fly, and it takes a few seconds of observations to make a fair prediction. This approach only applies to long-lived batch workloads, and makes less sense to interactive ones. 2) the outliers we eliminate in training

TABLE II
MAPE ERRORS ON TEST SET

Average Test Errors	**Image Processing**			**Video Processing**			**Linpack**		
	1s	3s	5s	1s	3s	5s	1s	3s	5s
raw	0.49	0.48	0.32	1.04	0.51	0.57	0.14	0.12	0.12
clean	0.52	0.47	0.31	1.00	0.57	0.57			
enhance	0.80	0.48	0.28	1.23	**0.49**	0.64	0.14	0.11	**0.11**
clean + enhance	0.81	0.47	**0.27**	5.60	0.54	0.61			

Best Model Errors	Linear Regressor			Gradient Boosting [5]			Linear Regressor		
	1s	3s	5s	1s	3s	5s	1s	3s	5s
raw	0.42	0.30	0.29	1.03	0.52	0.32	0.18	0.08	0.06
clean	0.44	0.30	0.29	0.97	0.57	**0.32**			
enhance	2.14	0.36	0.17	0.95	0.46	0.41	0.18	0.07	**0.05**
clean + enhance	2.05	0.36	**0.16**	0.87	0.47	0.45			

[a]Data cleaning is not effective for Linpack (no outliers identified), thus no result is present.

mostly result from platform-wise optimization. In other words, our estimation does not apply to those optimized invocations (˜30% in our experiments). Building an optimization-aware model might mitigate this problem.

IV. PREDICTION CONFIDENCE

Understandably, the execution time of different executions of the same serverless function with the same set of input parameters varies, as long as the variability keeps under a reasonable level. However, in practice, we find that the execution time of the same serverless function with the same set of input parameters could be distributed widely, especially under strictly available resources. This phenomenon is a fundamental problem because whatever model you use, you could only get one value, and the level of uncertainty still exists. To compensate for this issue, we instead try to give the scheduler prediction confidence, which indicates how stable the execution time distributes. In the first part of this section, we describe this phenomenon in more detail and offer profound insights. In the second part of this section, we state how we build a regression tree model to predict confidence.

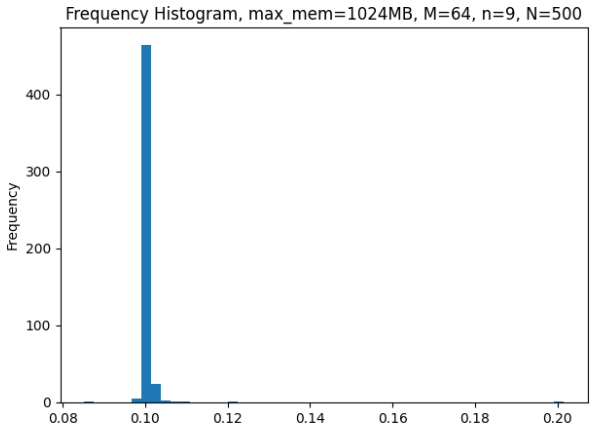


Fig. 4. Execution Time Distribution when Max Mem = 1024MB

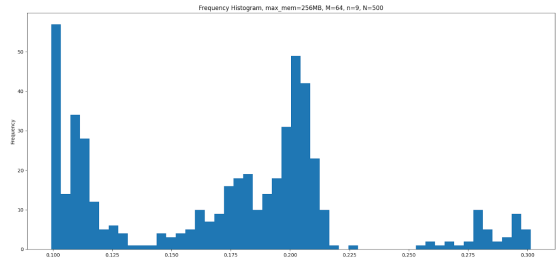


Fig. 5. Execution Time Distribution when Max Mem = 256MB

A. Insights from Experiments

We choose two symbolic figures to illustrate this phenomenon. Figures 4 and 5 are two execution time distribution graphs under the same serverless function with the same function parameters. This function is a cryptographic function, and M, n, N are the parameters for the cryptographic algorithm and has nothing to do with the problem here. The only difference is that figure 4 has a max memory capacity restriction of 1024MB, while figure 5 has a max memory capacity restriction of 256MB. We also see similar trends in other serverless function executions. We draw several insights about this as below.

1) Adding stricter resource limitations(in this case, the max memory capacity in the Google Cloud Platform) has minimal influence on the average execution time. However, it does influence the variance of the execution time.
2) The execution time distribution under limited resource is clearly clustered.
3) When the resource limitation is stricter, the difference between clusters is more significant, and the variance inside the cluster is also more considerable.

B. Prediction Confidence

To compensate for the effect, we decide to add another metric, the prediction confidence, along with the predicted ex-

ecution time. The prediction confidence is defined as follows:

$$\text{Prediction Confidence} = \frac{1}{max(\text{diff of the cluster average})}$$

We build a regression tree model to predict the prediction confidence. The input feature is memory peak usage, function parameters, and resource limitations. We treat 50 executions as a single training data point. We use 1000 training data points in total for one function. We run our model on three serverless functions: a linear programming function, a cryptographic function, and an image processing function. We achieve 5.0%, 7.6%, 8.1% error rate in average.

V. Potential Future Directions

In this section, we talk about the potential future directions of this project.

1) In this project, we only focused on building models to predict a single serverless function. Different serverless functions mean different models in our settings. In other words, each model is local to its related serverless function. However, there may be possible public features across different serverless functions. We plan to divide deeper into these "public" features and consider combining "local" models with "public" features.
2) In this project, we only focused on predicting the total execution time of a serverless function. We are also interested in other (predictable) resource requirements such as peak memory usage and storage usage.
3) In this project, we solely used the Google Cloud Platform for implementing our ideas and conducting all the related experiments. As with all the other major public cloud providers, the underlying implementing details remain a black box to us, which could hinder our understanding of the problem. In the future, we plan to run experiments on more platforms. We want to understand whether the insight we gain is platform-dependent or fundamental. We could also resort to some open-source platforms to have more control over what we are doing.
4) The more significant next step of the project is integrating the predicted value(such as total execution time, peak memory usage) and the corresponding pipeline into the scheduler and see the potential benefits of the improved resource utilization, among others, quantitively.

References

[1] Kim, J., & Lee, K. "Practical Cloud Workloads for Serverless Faas". In Proceedings of the ACM Symposium on Cloud Computing, pp. 477-477, November, 2019.

[2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. et al. "Scikit-learn: Machine learning in Python". the Journal of Machine Learning Research, 12, 2825-2830, November, 2011.

[3] Kuznetsova, A., Rom, H., Alldrin, N., Uijlings, J., Krasin, I., Pont-Tuset, J., et al. "The Open Images Dataset v4". International Journal of Computer Vision, 1-26, March, 2020.

[4] Kuehne, H., Jhuang, H., Garrote, E., Poggio, T., & Serre, T. "HMDB: a Large Video Database for Human Motion Recognition". In 2011 International Conference On Computer Vision, pp. 2556-2563. November, 2011.

[5] Friedman, J. H. "Greedy Function Approximation: a Gradient Boosting Machine". Annals of Statistics, 1189-1232, October, 1999.